PYTHON

02

Collections, Functions and Modules in Python

Master Python's core data structures and reusable code — the toolkit that turns a beginner into a productive programmer.

DURATION	LEVEL	MODULE	UPDATED
14 Sessions	Beginner	Module 2	May 2026

IN THIS MODULE

SESSION
01

60 MIN · COLLECTIONS
List Deep Dive

◆ Learning Objectives

- Create lists that hold mixed data types in a single variable
- Access any item using positive and negative indexing
- Update existing values inside a list
- Remove values using remove(), pop(), del and clear()
- Explain why a list is mutable and what that means in practice
- Apply list operations to a real student-record scenario

01 Why Lists Matter

A list is Python's way of keeping many values together under one name. Think of the attendance register at a TOPS batch in Ahmedabad — one register, many student names, all in a fixed order. A list works exactly like that register: one variable holds the whole class.

CONCEPT — What is a list

A list is an ordered, changeable collection written inside square brackets, like names = ['Ravi', 'Priya', 'Amit']. 'Ordered' means each item has a fixed position; 'changeable' means you can add, edit or delete items any time — just like marking a new student present in the register.

02 Creating Lists with Mixed Data

Unlike many other languages, a single Python list can hold different types at once — text, numbers, True/False, even another list. This is handy when one record has mixed details, like a student's name, age and fee-paid status together.

```
marks = [78, 92, 65, 88]                    A list of only numbers
student = ['Ravi', 21, True, 14999.0]       Mixed types: name, age, enrolled?, fee
empty = []                                  An empty list you can fill later
nested = ['Batch-A', ['Ravi', 'Priya']]     A list inside a list
```

TRY THIS LIVE

Make a list called my_profile with your name (text), your age (number) and whether you know Python yet (True/False). Then print(my_profile) and print(type(my_profile)).

03 Accessing & Updating Values

Every item has an index. Counting starts at 0 from the left, and -1 points to the last item from the right. Because lists are changeable, you can overwrite any item just by assigning a new value to its index.

```
names = ['Ravi', 'Priya', 'Amit']    Our list
print(names[0])                      First item -> Ravi
print(names[-1])                     Last item -> Amit
names[1] = 'Pooja'                   Update: Priya becomes Pooja
```

Method	What it does	Where used
append(x)	Adds x at the end	New student joins the batch
insert(i, x)	Adds x at position i	Insert a name in a fixed order
index(x)	Finds the position of x	Locate a student in the register
len(list)	Counts items	How many students enrolled

04 Removing Values

Python gives you three common ways to delete: remove() deletes by value, pop() deletes by position and hands the item back to you, and del removes by position without returning it. clear() empties the whole list.

```
names.remove('Amit')    Delete by name (value)
last = names.pop()      Remove last item AND keep it in 'last'
del names[0]            Delete the item at position 0
names.clear()           Empty the whole list
```

pop()

Removes by position and RETURNS the item, so you can use it next — like calling out a student before sending them home.

remove()

Removes by value and returns nothing. If the value is not in the list, it raises an error — so check first.

PRO TIP — Removing while looping is risky

Never delete items from a list while you are looping over that same list — Python loses count and skips items. Build a fresh list of what you want to keep instead. You will see why in the next session on iteration.

CASE STUDY — Lists power your Instagram feed

Context — Instagram runs one of the world's largest Python/Django backends, serving over two billion users.

Challenge — For every refresh, show each user an ordered, personalised set of posts chosen from a huge pool.

What they did — The feed is handled as an ordered collection — effectively a list of post items that the system builds, reorders and trims for each request.

Result — Billions of personalised feeds are assembled every day from list-style data structures.

Takeaway — Every time you scroll, you are watching an ordered list being built and updated in real time.

Source: Instagram Engineering (instagram-engineering.com)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms Using Python (Prof. Madhavan Mukund, CMI) — early lectures covering lists, indexing and mutability

Link — https://nptel.ac.in/courses/106106145

Always link the official channel — beware fan-made or re-upload copies.

"A list is just a register — one name on the cover, many lines inside."

WHERE TO GO NEXT

You can now store, read, update and delete data inside a list.

Explore deeper — Read the official Python list docs at docs.python.org/3/tutorial/datastructures.html

Try on your own — Build a list of your 5 favourite movies, then update one and remove another using three different methods.

Connect the dots — Next session we loop through these lists to process every item automatically.

Going further — Look up 'list slicing' — accessing a whole range like names[1:3] in one line.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a friendly Python tutor. Explain the difference between remove(), pop() and del on a Python list in simple Hinglish, using a college attendance register as the example. Keep it under 150 words and end with one quick practice question.

SESSION
02

60 MIN · COLLECTIONS
List Iteration Patterns

Learning Objectives

- Loop through every item of a list using a for loop
- Use range() to loop by position (index)
- Combine range() with len() to walk a list safely
- Use enumerate() to get position and value together
- Choose the right looping pattern for a given task
- Process a real list of student marks end to end

01 Looping Over a List

Doing the same action for every item is the most common job in programming. Imagine reading out all names from a TOPS attendance register one by one — a for loop does exactly that, automatically, no matter how long the list is.

CONCEPT — for loop over a list

A for loop visits each item in order and runs the same block of code for it. 'for name in names:' reads as 'for every name in the list names, do this' — like calling out each student until the register ends.

```
names = ['Ravi', 'Priya', 'Amit']      Our list
for name in names:                     Pick up each name, one at a time
    print('Present:', name)            Runs once per name (note the indent)
```

TRY THIS LIVE

Make a list of 5 fruits and loop over it to print 'I like ' + fruit for each one. Watch how the loop runs exactly 5 times without you counting.

02 Looping by Position with range()

Sometimes you need the position number, not just the value — for example to print 'Student 1, Student 2...'. range() gives you those numbers, and range(len(list)) gives one number for every item in the list.

```
for i in range(3):                     i becomes 0, then 1, then 2
    print(i)                           Prints 0 1 2 (stops before 3)
```

```
for i in range(len(names)):     One i for each item in names
    print(i, names[i])          Position AND value together
```

for item in list

Cleanest way when you only need the value. Most Pythonic. Use this by default.

for i in range(len(list))

Use only when you genuinely need the position number, or need to update items in place by index.

03 Best of Both — enumerate()

enumerate() is the clean way to get position and value at the same time, so you don't need range(len()). It hands you a counter and the item together on each turn of the loop.

```
for pos, name in enumerate(names):          pos = position, name = value
    print(pos + 1, name)                    Roll No. 1 Ravi, 2 Priya, ...
for pos, name in enumerate(names, start=1): Start counting from 1, not 0
    print(pos, name)                        1 Ravi, 2 Priya, 3 Amit
```

PRO TIP — reach for enumerate first

If you find yourself writing range(len(my_list)), pause. nine times out of ten enumerate(my_list) is cleaner and reads almost like English. Indian interviewers love seeing enumerate used well.

04 A Real Task — Process Marks

Let us put it together. Given a list of marks, loop through to find the total and the average — the exact logic behind any result sheet at a coaching class.

```
marks = [78, 92, 65, 88]          Marks of one student
total = 0                         Start the running total at zero
for m in marks:                   Visit each mark
    total = total + m             Add it to the running total
average = total / len(marks)      Total divided by how many marks
```

CASE STUDY — How Spotify chooses your next song

Context — Spotify, the global music app, uses Python heavily across its data and backend systems.

Challenge — Recommend songs by making sense of each user's long history of tracks played.

What they did — Their systems iterate over lists of listening events — going through each track one by one to spot patterns — the same loop idea you just learned, at massive scale.

Result — Personalised playlists like Discover Weekly are generated for hundreds of millions of users.

Takeaway — Every recommendation starts with a loop walking through a list of past actions.

Source: Spotify Engineering (engineering.atspotify.com)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms Using Python (Prof. Madhavan Mukund, CMI) — lectures covering for loops, range and iterating over lists

Link — https://nptel.ac.in/courses/106106145

"Write the logic once; let the loop repeat it a thousand times."

WHERE TO GO NEXT

You can now process every item in a list without copy-pasting code.

Explore deeper — Read about the 'else' clause on a for loop — a small Python feature most beginners miss.

Try on your own — Loop over a list of marks and count how many are 60 or above (a simple pass count).

Connect the dots — Next session we transform whole lists at once with sorted(), zip() and friends.

Going further — Explore list comprehension — doing a full loop in a single line. We cover it deeply later in this module.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a patient Python tutor. In simple Hinglish, explain the difference between 'for item in list' and 'for i in range(len(list))' using a cricket scoreboard as the example. Keep it under 150 words and give one tiny exercise at the end.

SESSION
03

60 MIN · COLLECTIONS
List Transformations

Learning Objectives

- Round numbers cleanly using round()
- Sort a list temporarily with sorted() without changing the original
- Sort a list permanently in place with .sort()
- Sort in reverse and by a custom key
- Pair up two lists item by item using zip()
- Combine these tools on a real price-and-product scenario

01 Cleaning Numbers with round()

Raw calculations often give long decimals like 78.66666. Before showing a result — say an average percentage on a marksheet — you round it so it reads cleanly. round() does exactly this.

```
round(78.66666)        No decimals -> 79
round(78.66666, 2)     Two decimal places -> 78.67
round(1499.5)          Rounds to the nearest even -> 1500
```

TRY THIS LIVE

Take a list of marks, find the average (total / count), and print it rounded to 2 decimal places — exactly how a real result sheet would show it.

02 sorted() vs .sort()

Both arrange a list in order, but there is one big difference. sorted() returns a NEW sorted list and leaves the original untouched. .sort() rearranges the ORIGINAL list itself and returns nothing. Pick based on whether you need to keep the original order.

sorted(list)

Makes a new sorted copy; original stays as it was. Use when you still need the unsorted data later.

list.sort()

Changes the original list permanently and returns None. Use when you no longer need the old order.

```
prices = [499, 199, 999, 299]      Original list
cheap_first = sorted(prices)       New list [199, 299, 499, 999]; prices unchanged
prices.sort()                      Now prices itself is sorted
prices.sort(reverse=True)          High to low -> [999, 499, 299, 199]
```

PRO TIP — a classic beginner bug

Writing my_list = my_list.sort() will make my_list become None, because .sort() returns nothing. If you want a result back, use sorted() instead. This trips up almost everyone once.

03 Sorting by a Custom Key

You can sort by a rule of your choice using key=. For example, sort a list of names by length, or sort words ignoring capital letters — useful for clean, human-friendly ordering.

```
names = ['Ravi', 'Pooja', 'Om', 'Krishna']   Names of mixed length
sorted(names, key=len)                       Shortest first -> ['Om', 'Ravi', 'Pooja', ...]
sorted(names, key=str.lower)                 A-Z ignoring upper/lower case
```

04 Pairing Lists with zip()

zip() locks two (or more) lists together item by item, like a zip on a jacket joining two sides tooth by tooth. It is the easiest way to walk a list of names alongside their marks at the same time.

```
names = ['Ravi', 'Priya', 'Amit']    List A
marks = [78, 92, 65]                 List B (same length)
for n, m in zip(names, marks):       Take one from each, side by side
    print(n, 'scored', m)            Ravi scored 78, Priya scored 92, ...
```

CONCEPT — zip stops at the shortest

If the two lists are different lengths, zip() stops at the shorter one and quietly ignores the rest. So always pair lists of equal length — like 3 names with exactly 3 marks.

CASE STUDY — The sort button you tap every day

Context — Python's built-in sort uses an algorithm called Timsort, created by developer Tim Peters in 2002.

Challenge — Sort huge real-world lists — product prices, search results — both quickly and reliably.

What they did — Timsort cleverly takes advantage of data that is already partly ordered, making it very fast on real data.

Result — It was so good it is now the default sort in Java, Android and the V8 engine that powers Chrome.

Takeaway — Every time you tap 'Sort by: Price — Low to High', you are using a world-class algorithm for free with one call to sorted().

Source: Python developer docs (github.com/python/cpython, listsort.txt) and Wikipedia: Timsort

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms Using Python (Prof. Madhavan Mukund, CMI) — lectures covering sorting and built-in list operations

Link — https://nptel.ac.in/courses/106106145

"sorted() keeps the original safe; .sort() rewrites it forever. Choose on purpose."

WHERE TO GO NEXT

You can now reshape, order and pair lists — the daily bread of data work.

Explore deeper — Read about the difference between sorting numbers and sorting strings (1, 2, 10 vs '1', '10', '2').

Try on your own — Pair a list of student names with their marks using zip(), then sort the pairs to find the topper.

Connect the dots — Next session we move from lists to tuples — the fixed, unchangeable cousin of the list.

Going further — Explore the 'key' parameter further with lambda functions to sort by complex rules.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a clear Python tutor. In simple Hinglish, explain the difference between sorted() and .sort() using a cricket batting-average table as the example. Show one line of code for each. Keep it under 150 words and end with a one-line tip.

SESSION
04

60 MIN · COLLECTIONS
Tuples — Advanced

Learning Objectives

- Create tuples that hold mixed data types
- Apply slicing rules to extract parts of a tuple
- Explain immutability and when it is an advantage
- Convert a tuple to a list and back again
- Use tuple packing and unpacking cleanly
- Choose between a list and a tuple for a given use case

01 What Makes a Tuple Different

A tuple looks like a list but uses round brackets and, most importantly, cannot be changed after creation. Think of it like a printed marksheet from a Gujarat university — once issued, the marks are fixed. You can read them, but you cannot edit them.

CONCEPT — Immutability

Immutable means the items inside a tuple cannot be updated, added or removed after the tuple is created. If you try, Python raises a TypeError. This is a feature, not a bug — it protects data that should never change, like a student's roll number or a GPS coordinate.

```
student = ('Ravi', 21, 'Ahmedabad')      Tuple with mixed types
coords = (23.0225, 72.5714)              Lat/long of Ahmedabad — should never change
single = (42,)                           A one-item tuple needs a trailing comma
student[0] = 'Pooja'                     TypeError! Tuples do not allow assignment
```

02 Creating Mixed Tuples

Like lists, tuples can mix types freely. A common real-world pattern is storing one complete record — name, ID, score — as a single tuple so all pieces stay together and cannot be accidentally overwritten.

```
record = ('Priya', 'TOPS-2025-042', 88.5, True)
    Name, ID, score, placed?
nested = ('Batch-A', ('Ravi', 91), ('Priya', 88))
    Tuple inside a tuple
empty = ()                                          An empty tuple
```

03 Slicing Tuples

Slicing works on tuples exactly as it does on lists and strings. You get a new tuple containing the items you picked. Slicing never modifies the original — which fits perfectly with immutability.

```
t = (10, 20, 30, 40, 50)      Our tuple
t[1:4]                        Items at index 1, 2, 3 -> (20, 30, 40)
t[:3]                         First three -> (10, 20, 30)
t[::2]                        Every second item -> (10, 30, 50)
t[::-1]                       Reversed -> (50, 40, 30, 20, 10)
```

TRY THIS LIVE

Create a tuple of your week's daily steps (7 numbers). Slice it to get just Mon–Wed, then slice again to get only the weekend days. Print both slices.

04 Tuple <-> List Conversion

When you receive a tuple but need to make changes, convert it to a list, do your edits, then convert back. This is the standard way to work around immutability when you genuinely need to.

```
marks_t = (78, 92, 65)        Original tuple
marks_l = list(marks_t)       Convert to list — now editable
marks_l.append(88)            Add a new mark
marks_t = tuple(marks_l)      Convert back to a fixed tuple
```

05 Packing & Unpacking

Tuple unpacking lets you split a tuple's values into separate named variables in one clean line. Python functions often return multiple values this way — you have already seen it with enumerate() pairing position and item.

```
student = ('Ravi', 21, 'Ahmedabad')    One tuple, three pieces
name, age, city = student              Unpack all three at once
print(name, age, city)                 Ravi 21 Ahmedabad
first, *rest = (10, 20, 30, 40)        first=10, rest=[20,30,40] — starred unpacking
```

List

Mutable — add, remove, update freely. Use for collections that change over time, like a batch's attendance.

Tuple

Immutable — fixed after creation. Use for data that must not change, like coordinates, records, or function return values.

PRO TIP — Tuples as dictionary keys

Because tuples are immutable, Python trusts them as dictionary keys — lists are not allowed. This is why (city, pincode) pairs work as map lookup keys. You will use this in Session 5.

CASE STUDY — Why Python has both lists and tuples

Context — Guido van Rossum and the Python design team deliberately gave Python two sequence types.

Challenge — Some data in a program should be locked — guaranteed not to change anywhere in the code — while other data needs to grow and shrink freely.

What they did — They made tuples immutable: once created, no code can alter their contents. This also allows Python to store tuples more compactly in memory and use them as dictionary keys.

Result — Tuples are slightly faster to create and take less memory than equivalent lists, and they carry a clear signal to any reader: this data is fixed.

Takeaway — Choose a tuple when the data represents a fixed record; choose a list when the data will change.

Source: Python documentation (docs.python.org/3/tutorial/datastructures.html) and Python FAQ

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms Using Python (Prof. Madhavan Mukund, CMI) — lectures covering tuples, immutability and sequence types

Link — https://nptel.ac.in/courses/106106145

“A list is a notebook you can edit. A tuple is a printed certificate — read only.”

WHERE TO GO NEXT

You now have a clear mental model of when to use a tuple versus a list.

Explore deeper — Read about named tuples (collections.namedtuple) — a way to give each position a

name like a mini object.

Try on your own — Store your home address as a tuple (city, area, pincode). Try to change the pincode — read the error, then do it the right way via list conversion.

Connect the dots — Next session we move to dictionaries — key-value pairs that let you look up data by name, not position.

Going further — Look into how tuples make functions able to return multiple values cleanly — a pattern used everywhere in Python libraries.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. In simple Hinglish, explain why Python has both lists and tuples, using a printed marksheet versus a rough notebook as the analogy. Keep it under 150 words and end with one real use case of each.

SESSION
05

60 MIN · COLLECTIONS
Dictionaries Deep Dive

◆ Learning Objectives

- Access dictionary values using their keys
- Add new key-value pairs and update existing ones safely
- Use .get() to avoid KeyError crashes
- Build and access nested dictionaries
- Iterate over a dictionary using .keys(), .values() and .items()
- Apply dictionary operations to a real student-record scenario

01 Quick Recap — What a Dictionary Is

A dictionary stores data as key-value pairs inside curly braces. Instead of a position number, you look up values by name. Think of an Aadhaar card — you do not ask for 'field number 3', you ask for 'address'. That is exactly how a Python dictionary works.

```
student = {'name': 'Ravi', 'age': 21, 'city': 'Ahmedabad'}
    A simple dictionary
student['name']                          Access by key -> 'Ravi'
student.get('phone')                     Key missing? Returns None instead of crashing
student.get('phone', 'Not found')        Missing key with a safe default value
```

CONCEPT — get() vs direct access

student['phone'] crashes with a KeyError if the key does not exist. student.get('phone') returns None quietly. Always use .get() when you are not sure whether a key is present — real data is always messy.

02 Adding & Updating Values

Dictionaries are mutable — you can add new keys or change existing values any time. Adding a key that does not exist yet is the same syntax as updating one that does — Python decides automatically.

```
student['email'] = 'ravi@example.com'          Add a brand new key
student['age'] = 22                            Update an existing key
student.update({'city': 'Surat', 'score': 88})
    Update multiple keys at once
del student['score']                           Remove a key entirely
```

```
student.pop('city')                    Remove and return the value
```

TRY THIS LIVE

Build a dictionary for yourself — name, city, course, batch year. Then update your city, add a 'skills' key with a list as its value, and print the whole dictionary.

03 Iterating Over a Dictionary

Python gives you three clean ways to loop over a dictionary — by keys only, by values only, or by both at the same time. .items() is the most useful in practice because you rarely want one without the other.

```
for key in student.keys():              Loop through key names
    print(key)                          name, age, city, email
for val in student.values():            Loop through values
    print(val)                          Ravi, 22, Ahmedabad, ...
for key, val in student.items():        Both together (most useful)
    print(key, ':', val)                name : Ravi | age : 22 | ...
```

Method	Returns	Best used when
.keys()	All keys	You only need the field names
.values()	All values	You need to sum or compare data
.items()	Key-value pairs	You need both — the most common loop

04 Nested Dictionaries

A dictionary value can itself be another dictionary. This is called nesting. It is the natural way to represent structured real-world records — like a batch of students where each student has multiple fields.

```
batch = {                                   Outer dictionary — batch name as key
'Ravi':  {'age': 21, 'score': 88, 'city': 'Ahmedabad'},
   Ravi's inner record
'Priya': {'age': 22, 'score': 92, 'city': 'Surat'}
   Priya's inner record
}
```

```
batch['Ravi']['score']                  Drill in: outer key -> inner key -> 88
batch['Priya']['city'] = 'Vadodara'     Update a nested value
```

PRO TIP — setdefault() for safe nesting

When building nested dicts dynamically, batch.setdefault('NewStudent', {}) creates the inner dict only if it does not exist yet — it will not overwrite an existing one. Far safer than a plain assignment inside a loop.

student['phone']

Crashes with KeyError if 'phone' is not in the dict. Use only when you are 100% sure the key exists.

student.get('phone')

Returns None (or your default) if the key is missing. Always safer when working with real-world data.

CASE STUDY — Every API response is a dictionary

Context — Apps like Swiggy, MakeMyTrip and the OpenWeatherMap API all send data in JSON format.

Challenge — Send structured data — restaurant details, flight info, weather readings — over the internet in a format any language can read.

What they did — They use JSON (JavaScript Object Notation), which maps directly onto Python dictionaries. Python's built-in json module converts a JSON response into a Python dict in one line: data = json.loads(response.text).

Result — Every Python developer who works with web APIs works with dictionaries daily — the two are inseparable.

Takeaway — Mastering nested dictionaries is the same skill as reading real API responses.

Source: Python docs — json module (docs.python.org/3/library/json.html)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms Using Python (Prof. Madhavan Mukund, CMI) — lectures covering dictionaries, key-value access and nested structures

Link — https://nptel.ac.in/courses/106106145

"A dictionary is the closest Python gets to the real world — you look things up by name, not by number."

WHERE TO GO NEXT

You can now build, update, delete and deeply access dictionary data.

Explore deeper — Read about collections.defaultdict — a dictionary that automatically creates a default value for any missing key.

Try on your own — Build a nested dictionary for a class of 3 students, each with name, marks and city. Loop over it with .items() to print a formatted result card for each.

Connect the dots — Next session we create dictionaries from lists — pairing names with values automatically using zip() and loops.

Going further — Look into dictionary comprehensions — building a whole dict in one concise line, similar to list comprehensions.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. In simple Hinglish, explain nested dictionaries using an Aadhaar card that also stores a person's family members as the example. Keep it under 150 words and show one short code snippet.

SESSION
06

60 MIN · COLLECTIONS

Converting Lists to Dictionaries

Learning Objectives

- Convert two parallel lists into a dictionary using zip()
- Use dict() with zip() as a single clean line
- Build a dictionary from lists using a for loop with manual control
- Choose between zip() and a loop for a given task
- Apply both methods to real student and CSV-style data
- Recognise this pattern in real data pipelines

01 Why Convert Lists to Dictionaries?

Raw data often arrives as two separate lists — one of field names, one of values. A TOPS student export might give you ['name', 'score', 'city'] and ['Ravi', 88, 'Ahmedabad'] as two rows. To work with that data meaningfully, you need to pair them into a dictionary so you can look up 'score' by name, not by position number.

CONCEPT — Two lists, one dictionary

If you have a keys list and a values list of the same length, Python can zip them together like teeth on a jacket zip. dict(zip(keys, values)) turns both lists into one lookup-friendly dictionary in a single line.

02 Method 1 — dict(zip())

This is the fastest and most Pythonic way. zip() pairs up items by position, then dict() converts those pairs into key-value entries. It reads almost like plain English once you know what zip does.

```
fields = ['name', 'score', 'city']      Keys list — like CSV column headers
values = ['Ravi', 88, 'Ahmedabad']      Values list — one student's row
student = dict(zip(fields, values))     Combine -> {'name':'Ravi','score':88,...}
print(student['score'])                 Now look up by name, not position -> 88
```

TRY THIS LIVE

Write two lists — one with 4 subject names, one with 4 marks. Use dict(zip()) to combine them. Then print each subject and its mark by looping over .items().

03 Method 2 — Building with a for Loop

When you need more control — for example to skip certain pairs, transform values, or add conditions — a for loop gives you full flexibility. You build the dictionary item by item inside the loop.

```
fields = ['name', 'score', 'city']      Keys
values = ['Ravi', 88, 'Ahmedabad']      Values
student = {}                            Start with an empty dictionary
for key, val in zip(fields, values):    Walk both lists together
    student[key] = val                  Add each pair manually
```

The loop approach also lets you add logic — for example only storing a key if the value is not empty, or capitalising names as you insert them.

```
for key, val in zip(fields, values):    Same loop, now with a condition
    if val != '':                       Skip blank values (common in real exports)
        student[key] = val              Only store non-empty fields
```

dict(zip(keys, values))

One line, clean, fast. Use when all pairs go in as-is with no conditions or transformations.

for loop + zip

More code, full control. Use when you need to filter, transform or validate each pair before storing it.

04 Many Rows — A List of Dictionaries

Real data has many records, not just one. The standard pattern is a list of dictionaries — one dictionary per row, all sharing the same keys. This is exactly what you get when you load a CSV with Python's csv.DictReader, or receive a list of records from an API.

```
fields = ['name', 'score', 'city']                  Column headers
rows = [['Ravi',88,'Ahmedabad'],['Priya',92,'Surat']]
   Each row is a values list
students = []                                       Will hold one dict per student
for row in rows:                                    Process every row
    students.append(dict(zip(fields, row)))         Convert and add to the list
print(students[0]['name'])                          Ravi
```

PRO TIP — This is how csv.DictReader works

Python's built-in csv.DictReader does exactly this pattern for you — it reads the first row as field names and converts every following row into a dictionary. Understanding zip() and this loop means you already understand how DictReader works under the hood.

CASE STUDY — Spreadsheet data comes to life as dictionaries

Context — Every company that processes tabular data in Python — from small startups to Zerodha handling millions of trade records — works with CSV or Excel files.

Challenge — A raw CSV file is just rows of values with no context. Code that accesses row[2] is fragile — move a column and everything breaks.

What they did — Python's csv.DictReader (and pandas) uses the header row as keys and converts each data row into a dictionary. Internally it zips the header list with each data row — exactly the pattern you learned today.

Result — Code accesses data by meaningful names like row['price'] instead of fragile positions like row[3], making it far more readable and less error-prone.

Takeaway — The dict-zip pattern is the foundation of almost all real-world data loading in Python.

Source: Python docs — csv module (docs.python.org/3/library/csv.html)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms Using Python (Prof. Madhavan Mukund, CMI) — lectures covering dictionaries and zip-based data pairing

Link — https://nptel.ac.in/courses/106106145

"Two plain lists plus zip equals one smart dictionary — that is half of all real data loading in Python."

WHERE TO GO NEXT

You can now turn any two parallel lists into a lookup-ready dictionary.

Explore deeper — Read Python's csv.DictReader documentation — you will immediately recognise the pattern you built today.

Try on your own — Create a CSV-style dataset in your code (a list of rows). Convert every row into a dictionary and print the name and score of any student whose score is 80 or above.

Connect the dots — Next session we use this kind of dictionary to count things — building a frequency map of characters and words.

Going further — Explore dictionary comprehensions — a one-line way to build a dict from a loop: {k: v for k, v in zip(keys, values)}.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. In simple Hinglish, explain how dict(zip(keys, values)) works using a student report card as the example — one list for subject names, one for marks. Keep it under 150 words and show the code output clearly.

SESSION
07

60 MIN · COLLECTIONS

Counting Characters & Words

Learning Objectives

- Build a frequency dictionary that counts occurrences of each character
- Use .get() and the get-then-add pattern to count safely
- Count every word in a sentence using .split() and a freq dictionary
- Find the most and least common item from a frequency map
- Use collections.Counter as the professional shortcut
- Apply a word-count program to a real text-processing task

01 The Frequency Dictionary Pattern

A frequency dictionary maps each unique item to how many times it appears. This one pattern powers spam filters, search engines, autocorrect and plagiarism checkers. The logic is always the same: loop over the data, and for each item add 1 to its count in the dictionary.

CONCEPT — get-then-add

The core counting pattern is: freq[item] = freq.get(item, 0) + 1. Read it as: 'look up the current count; if the key is missing, start from 0; then add 1.' This one line is all you need to count anything.

02 Counting Characters

Strings are sequences — looping over a string visits one character at a time, exactly like looping over a list. That makes character counting a direct application of everything you have learned so far.

```
word = 'Ahmedabad'                  Our test string
freq = {}                           Start with an empty dictionary
for ch in word:                     Visit each character one at a time
    freq[ch] = freq.get(ch, 0) + 1  Add 1 to that character's count
print(freq)                         {'A':1, 'h':1, 'm':1, 'e':1, 'd':2, 'a':2, 'b':1}
```

TRY THIS LIVE

Count the characters in your own first name. Then find which character appears the most by looping over the frequency dictionary and tracking the maximum count.

03 Counting Words

To count words, split the sentence into a list of words first using .split(), then apply the same get-then-add pattern. .split() without arguments handles extra spaces automatically — very useful for messy real input.

```
sentence = 'ravi likes python and priya also likes python'
   Our test sentence
words = sentence.split()                    ['ravi','likes','python','and','priya','also','likes','python']
freq = {}                                   Empty frequency dictionary
for w in words:                             Loop over each word
    freq[w] = freq.get(w, 0) + 1            Same pattern as character counting
print(freq)                                 {'ravi':1,'likes':2,'python':2,'and':1,...}
```

04 Finding the Most Common Item

Once you have a frequency dictionary, finding the most or least common item is a matter of sorting by value. Use max() with key=freq.get to get the winner in one line.

```
top_word = max(freq, key=freq.get)          Key with the highest value -> 'python'
low_word = min(freq, key=freq.get)          Key with the lowest value
sorted_freq = sorted(freq, key=freq.get, reverse=True)
   All keys sorted by count, high to low
```

05 The Professional Shortcut — Counter

Python's collections.Counter does everything above automatically. It builds the frequency dictionary for you and adds a most_common() method. In real projects you will use Counter, but building it from scratch first means you understand exactly what is happening inside.

```
from collections import Counter     Import from the standard library
freq = Counter(words)               Builds the full frequency dict instantly
freq.most_common(3)                 Top 3 words and their counts
freq['python']                      Count for a specific word -> 2
```

> **PRO TIP — Normalise before counting**
>
> Real text is messy. 'Python', 'python' and 'PYTHON' will be counted as three different words unless you normalise first. Always apply .lower().strip() to each word before adding it to the frequency dictionary.

Task	One-liner	Notes
Build freq dict	Counter(items)	Works on any iterable
Top N items	Counter.most_common(N)	Returns list of (item, count) tuples
Most common item	max(freq, key=freq.get)	Works on a plain dict too
Normalise text	word.lower().strip()	Do this before counting

CASE STUDY — How Gmail's spam filter learned to read

Context — In 2002, programmer Paul Graham published a technique that transformed email spam filtering.

Challenge — Automatically separate spam from real emails without hand-writing thousands of rules.

What they did — He counted word frequencies in confirmed spam emails versus real emails, building a probability score for each word. Words like 'guaranteed' appeared far more in spam; words like 'meeting' appeared more in genuine emails.

Result — The technique, called Bayesian spam filtering, became the foundation of modern spam detection and is still used by Gmail and others today.

Takeaway — A frequency dictionary of words is not a beginner exercise — it is the foundation of real machine intelligence.

Source: Paul Graham, 'A Plan for Spam' (paulgraham.com/spam.html), 2002

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms Using Python (Prof. Madhavan Mukund, CMI) — lectures covering dictionaries and string processing

Link — https://nptel.ac.in/courses/106106145

“Count what happens; the pattern in the numbers tells you everything.”

WHERE TO GO NEXT

You can now turn any text into structured frequency data — the entry point to text analytics.

Explore deeper — Read Paul Graham's original 'A Plan for Spam' article — it is short, clear and shows word counting in action at its most impactful.

Try on your own — Take any paragraph from a news article, count the word frequencies and find the top 5 words. Then strip out common words ('the', 'a', 'is') and see how the list changes.

Connect the dots — Next session we revisit functions with a fresh focus on positional and keyword arguments — the building blocks of reusable, flexible code.

Going further — Explore collections.Counter's arithmetic — you can add two Counters together to merge frequency data from multiple sources.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. In simple Hinglish, explain how to build a word frequency dictionary using the get-then-add pattern, with the example of counting words in a Bollywood movie review. Keep it under 150 words and show the key code line clearly.

SESSION
08

60 MIN · FUNCTIONS
Functions Revisited

Learning Objectives

- Distinguish between positional and keyword arguments
- Call a function using keyword arguments in any order
- Define default values so some arguments become optional
- Understand the rule: non-default arguments must come before default ones
- Use *args to accept any number of positional arguments
- Design a clean, reusable function for a real student-record task

01 Positional Arguments

In a basic function call, arguments are matched to parameters by their position. The first value goes to the first parameter, the second to the second, and so on. Order matters completely — swap two values and the function receives them wrongly.

```
def greet(name, city):                    Two positional parameters
    print(f'Hello {name} from {city}!')
greet('Ravi', 'Ahmedabad')                Correct -> Hello Ravi from Ahmedabad!
greet('Ahmedabad', 'Ravi')                Wrong order -> Hello Ahmedabad from Ravi!
```

02 Keyword Arguments

Keyword arguments let you name each value at the call site, so the order no longer matters. This makes function calls much easier to read and far less prone to bugs when a function has many parameters.

```
greet(city='Surat', name='Priya')         Reversed order — still works perfectly
greet(name='Amit', city='Vadodara')       Both named — completely clear
```

CONCEPT — Positional vs Keyword

Positional: order decides which value goes where. Keyword: name decides, order is free. You can mix them — but positional arguments must always come before keyword arguments in the call.

TRY THIS LIVE

Write a function create_profile(name, course, city) and call it three ways: all positional, all keyword, and mixed. Confirm all three give the same output.

03 Default Values

A default value makes a parameter optional — the caller can skip it, and the function uses the preset value instead. This is what lets you call print() without specifying end='' every time; Python already knows the default is a newline.

```
def enroll(name, course='Python', city='Ahmedabad'):
    Two params have defaults
print(f'{name} enrolled in {course} at {city}')
enroll('Ravi')                          Ravi enrolled in Python at Ahmedabad
enroll('Priya', 'Django')               Priya enrolled in Django at Ahmedabad
enroll('Amit', city='Surat')            Amit enrolled in Python at Surat
```

WARNING — Default order rule

Parameters with default values must always come AFTER parameters without defaults. Writing def enroll(course='Python', name) will raise a SyntaxError. Python needs to be able to fill in positional arguments unambiguously.

04 *args — Any Number of Positional Arguments

Sometimes you do not know how many values a caller will pass. The *args syntax collects all extra positional arguments into a tuple, so the function can handle one, five or fifty values without changing its definition.

```
def total_marks(*marks):            marks is a tuple of however many values are passed
    return sum(marks)               sum() works on any iterable
total_marks(78, 92)                 Works -> 170
total_marks(78, 92, 65, 88, 74)     Still works -> 397
```

Fixed parameters

def add(a, b) — caller must always pass exactly two values. Simple and explicit; use when the count is always the same.

*args

def add(*nums) — caller can pass any number of values. Use when the count varies, like summing a flexible list of marks.

05 Putting It Together — A Real Function

Good function design uses default values for common cases and keyword arguments for clarity, so callers only specify what is genuinely different for their situation.

```
def student_report(name, *marks, grade='B', city='Ahmedabad'):
    Mixed: required, *args, keyword-only defaults
    avg = sum(marks) / len(marks)                Average of all marks passed
print(f'{name} | Avg: {avg:.1f} | Grade: {grade} | {city}')
student_report('Ravi', 78, 92, 65, grade='A')    Ravi | Avg: 78.3 | Grade: A | Ahmedabad
```

PRO TIP — Mutable default values are a trap

Never use a list or dict as a default value: def f(data=[]). Python creates that list once and reuses it for every call — changes in one call bleed into the next. Use def f(data=None) and create the list inside the function if needed. This is one of the most famous Python gotchas.

CASE STUDY — How the 'requests' library stays easy to use

Context — The requests library by Kenneth Reitz is one of the most downloaded Python packages in the world, used by millions of developers.

Challenge — Fetching a web page involves many options — headers, timeouts, authentication, proxies — but most users only need the URL.

What they did — Every optional parameter has a sensible default value: timeout=None, headers=None, verify=True. Callers specify only what they actually need to change.

Result — The simplest call is just requests.get(url) — one argument. Advanced users can layer in as many keyword arguments as they need without the simple case becoming complex.

Takeaway — Thoughtful default values are what turn a powerful function into a friendly one.

Source: requests library documentation (docs.python-requests.org)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms Using Python (Prof. Madhavan Mukund, CMI) — lectures covering functions, parameters and argument passing

Link — https://nptel.ac.in/courses/106106145

“A good default value means the caller only needs to say what is different — everything else is already handled.”

WHERE TO GO NEXT

You can now write functions that are flexible, safe and easy to call.

Explore deeper — Look up **kwargs — the counterpart to *args that collects keyword arguments into a dictionary. Together they make a function that accepts absolutely anything.

Try on your own — Write a function send_sms(to, message, sender='TOPS-INFO') that prints a formatted message. Call it with different combinations of positional and keyword arguments.

Connect the dots — Next session we look at anonymous functions — lambdas — for short, throwaway operations that do not need a full def.

Going further — Read about Python's inspect module, which lets you examine a function's parameters and defaults programmatically — useful for building flexible frameworks.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. In simple Hinglish, explain the difference between positional arguments, keyword arguments and default values using a chai order at a Gujarat tapri as the example. Keep it under 150 words and end with one practical rule to remember.

SESSION
09

60 MIN · FUNCTIONS

Anonymous Functions

Learning Objectives

- Write a lambda function and explain what makes it anonymous
- Use lambda as the key= argument in sorted()
- Apply map() to transform every item in a list
- Apply filter() to keep only items that pass a test
- Use reduce() from functools to collapse a list to a single value
- Decide when to use a lambda versus a named function

01 What is a Lambda?

A lambda is a tiny, nameless function written in one line. You use it when you need a small operation just once — perhaps as an argument to sorted() or map() — and it is not worth writing a full def block for. The word 'anonymous' simply means it has no name.

CONCEPT — Lambda syntax

lambda arguments : expression. Everything to the left of the colon is input; everything to the right is what gets returned. There is no return keyword — the expression result is always returned automatically. lambda x: x * 2 is identical to def double(x): return x * 2.

```
double = lambda x: x * 2
double(5)
add = lambda a, b: a + b
add(3, 7)
classify = lambda n: 'pass' if n >= 35 else 'fail'
```

Assigned to a name (unusual, but shows the idea)
-> 10
Two inputs
-> 10
Conditional expression inside lambda

PRO TIP — Do not assign lambdas to names

Writing double = lambda x: x * 2 and then reusing 'double' everywhere defeats the purpose of an anonymous function. If you need to reuse it, write a proper def. Lambdas earn their keep only when used inline as a one-time argument.

02 Lambda with sorted()

The most common real use of lambda is as the key= argument in sorted(). Instead of defining a separate function just to describe the sorting rule, you write the rule inline in one line.

```
students = [('Ravi',88), ('Priya',92), ('Amit',65)]
    List of (name, score) tuples
sorted(students, key=lambda s: s[1])              Sort by score (index 1), low to high
sorted(students, key=lambda s: s[1], reverse=True)
    Sort by score, high to low
sorted(students, key=lambda s: s[0])              Sort alphabetically by name (index 0)
```

TRY THIS LIVE

Create a list of 5 product tuples (name, price). Sort them lowest price first, then highest price first, using lambda each time.

03 map() — Transform Every Item

map() applies a function to every item in a list and returns the transformed results. Combined with lambda, it is a compact way to reshape a whole list in one expression — equivalent to a for loop that builds a new list.

```
marks = [78, 92, 65, 88]                          Original marks
curved = list(map(lambda m: m + 5, marks))        Add 5 to every mark -> [83, 97, 70, 93]
names = ['ravi', 'priya', 'amit']                 Lowercase names
proper = list(map(lambda n: n.capitalize(), names))
    ['Ravi', 'Priya', 'Amit']
```

04 filter() — Keep Only What Passes

filter() keeps only the items for which the function returns True. Think of it as a sieve — data goes in, only the matching pieces come out. Combined with lambda, filtering a list by any condition is one line of code.

```
marks = [78, 32, 92, 45, 65, 28, 88]              Mixed pass/fail marks
passed = list(filter(lambda m: m >= 35, marks))
    Keep only passing marks -> [78, 92, 45, 65, 88]
names = ['Ravi', 'Pooja', 'Raj', 'Priya', 'Om']
    Names list
```

```
long_names = list(filter(lambda n: len(n) > 3, names))
  ['Ravi', 'Pooja', 'Priya']
```

map()

Transforms every item — the output list is the same length as the input. Use to change each value (add 5, capitalise, convert units).

filter()

Keeps only matching items — the output list is the same length or shorter. Use to select a subset (passing marks, active users, non-empty strings).

05 reduce() — Collapse to One Value

reduce() folds a list down to a single value by repeatedly applying a two-argument function: take the running result and the next item, combine them, repeat. Unlike map() and filter(), reduce() lives in the functools module.

```
from functools import reduce                    Must import explicitly
marks = [78, 92, 65, 88]
total = reduce(lambda a, b: a + b, marks)       78+92=170, 170+65=235, 235+88=323
highest = reduce(lambda a, b: a if a > b else b, marks)
  Keep whichever is larger each step -> 92
```

CONCEPT — map, filter, reduce together

These three form a classic data-processing pipeline. filter selects the right items, map transforms them, reduce summarises them. Example: from a list of all marks, filter passing students, map to add a 5-point bonus, reduce to find the highest score after the bonus.

CASE STUDY — Lambda gave its name to cloud computing

Context — AWS Lambda, launched by Amazon in 2014, is one of the world's most widely used cloud services.

Challenge — Run code in the cloud without managing a server — triggered by events, running only for the seconds it is needed.

What they did — Amazon named the service after the concept of anonymous functions in computer science: small, self-contained units of logic that execute on demand and return a result — exactly like a lambda.

Result — AWS Lambda now runs trillions of function invocations per month for companies worldwide, including startups across India.

Takeaway — The lambda concept you are learning today is so fundamental that a billion-dollar cloud platform is named after it.

Source: AWS Lambda documentation and announcement (aws.amazon.com/lambda)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms Using Python (Prof. Madhavan Mukund, CMI) — lectures covering functions and higher-order programming

Link — https://nptel.ac.in/courses/106106145

"Lambda: a function so small it does not need a name — but powerful enough to name a cloud platform after."

WHERE TO GO NEXT

You can now write concise transformations, filters and reductions without a full def block.

Explore deeper — Read about list comprehensions — they often replace map() and filter() with even cleaner syntax. We cover these in depth in Session 13.

Try on your own — Take a list of 10 product prices. Use filter() to keep only those above 500, map() to apply a 10% discount, and reduce() to find the total after discount.

Connect the dots — Next session we write and import our own custom modules — the skill that turns a single script into a structured project.

Going further — Explore functools.partial — a way to freeze some arguments of a function and create a specialised version of it, like a pre-configured lambda.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. In simple Hinglish, explain map(), filter() and reduce() using a vegetable mandi as the example — filter keeps only fresh vegetables, map converts prices from per kg to per 500g, and reduce totals the bill. Keep it under 150 words and give one line of code for each.

SESSION
10

60 MIN · FUNCTIONS
Modules — Custom

Learning Objectives

- Explain what a module is and why splitting code into files matters
- Create a custom module by saving functions in a separate .py file
- Import a module using import, from...import and import...as
- Use __name__ == '__main__' to protect module-level code from running on import
- Organise a multi-file project with a clear folder structure
- Identify how this pattern appears in every real Python framework

01 Why Modules Exist

When a project grows, keeping all code in one file becomes unmanageable — like storing every document of a company in a single drawer. A module is simply a .py file containing functions, variables and classes that other files can borrow. Split your code by responsibility, and each piece becomes easier to find, test and reuse.

> **CONCEPT — What a module is**
>
> Any .py file is automatically a module. If you save functions in student.py, another file can access them with import student. There is nothing to install or configure — Python picks up any .py file in the same folder.

02 Creating Your First Module

Start by writing helper functions in a dedicated file. A clean convention: name the file after what it does — student.py holds student-related functions, calc.py holds calculation functions. Keep each module focused on one responsibility.

student.py — the module file

```
# student.py                              This file IS the module
def greet(name):                          A simple greeting function
    return f'Welcome to TOPS, {name}!'
def average(marks):                       Calculates average of a marks list
    return sum(marks) / len(marks)
BATCH = 'Morning'                         A module-level constant
```

main.py — the file that uses the module

```
import student                    Import the whole module
print(student.greet('Ravi'))      Access with dot notation
print(student.BATCH)              Access a module variable too
```

TRY THIS LIVE

Create student.py with greet() and average(). Then create main.py in the same folder, import student, and call both functions. Run main.py and confirm the output.

03 Three Ways to Import

Python gives you three import styles. Each has its place — the right choice depends on how many things you need from the module and how much you want to type each time you use them.

```
import student                          Import whole module; use student.greet() each time
from student import greet, average      Import specific names; call greet() directly
from student import *                   Import everything — avoid in real projects (pollutes namespace)
import student as st                    Alias for a shorter name; use st.greet()
```

Style	Best used when	Downside
import module	Using many things from the module	More typing each call
from module import x	Using one or two specific names	Can clash with a local name
import module as alias	Module name is long or conflicts	Team must know the alias
from module import *	Quick scripts only	Pollutes namespace; avoid in projects

04 Protecting Code with __name__

When Python imports a module it runs all top-level code in that file. If student.py has a print() at the bottom, it will print every time someone imports it — which is never what you want. The __name__ guard fixes this.

```
# Inside student.py
def greet(name):                        Your function — always importable
    return f'Welcome, {name}!'
if __name__ == '__main__':              This block runs ONLY when you run student.py directly
    print(greet('Test User'))           Safe test — skipped on import
```

PRO TIP — Always use the __name__ guard

Every .py file you write that contains runnable test code should have the if __name__ == '__main__': guard. It is the single most consistent habit that separates clean Python files from messy ones. Django, NumPy and every serious Python project uses it.

Running directly

python student.py — Python sets __name__ to '__main__', so the guarded block runs. Good for quick testing.

Importing

import student from another file — Python sets __name__ to 'student', so the guarded block is skipped. Only the functions are loaded.

CASE STUDY — Django is just well-organised modules

Context — Django, the Python web framework you will use in Module 4, powers Instagram, Disqus and thousands of Indian startups.

Challenge — A web framework must handle routing, templates, databases, forms, authentication and more — far too much for one file.

What they did — Django is split into focused modules: django.urls, django.template, django.db, django.forms, django.contrib.auth. Each does one job. You import only what you need for any given task.

Result — A developer can build a full web application by composing focused modules — the same skill you are practising today with student.py.

Takeaway — Professional Python development is the art of organising code into modules that can be composed and reused.

Source: Django documentation (docs.djangoproject.com/en/stable)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms Using Python (Prof. Madhavan Mukund, CMI) — lectures covering modules, import system and code organisation

Link — https://nptel.ac.in/courses/106106145

“One file for one job — that is the habit that turns a script into a professional project.”

WHERE TO GO NEXT

You can now split any project into clean, importable .py files.

Explore deeper — Read about Python's module search path (sys.path) — it explains why Python can find your module and what to do when it cannot.

Try on your own — Create a calc.py module with add(), subtract(), multiply() and average(). Import and use it from main.py without copying any code between files.

Connect the dots — Next session we explore Python's built-in math and random modules — professional tools that are ready to import right now.

Going further — Read about Python's importlib — the machinery behind import statements — for a deeper understanding of how the import system works.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. In simple Hinglish, explain what a Python module is and why we split code into separate files, using a TOPS institute with separate departments (admissions, training, placement) as the analogy. Keep it under 150 words.

SESSION
11

60 MIN · FUNCTIONS

math & random Modules

Learning Objectives

- Import and use functions from Python's built-in math module
- Apply sqrt(), ceil(), floor(), pow() and log() to real problems
- Use math.pi and math.e as precise constants
- Generate random integers and floats with the random module
- Shuffle a list and pick a random sample using random
- Seed the random generator for reproducible results

01 The math Module

Python's math module wraps the highly optimised C math library and gives you accurate, fast mathematical functions without writing them yourself. From calculating EMI interest to finding a distance on a map, math has a ready-made tool.

CONCEPT — Importing a standard library module

Standard library modules come installed with Python — no pip needed. import math loads the module; then every function is accessed as math.sqrt(), math.ceil(), and so on. You can also write from math import sqrt, pi to use them without the math. prefix.

```
import math              Load the module
math.sqrt(144)           Square root -> 12.0
math.ceil(4.2)           Round UP to nearest integer -> 5
math.floor(4.9)          Round DOWN to nearest integer -> 4
math.pow(2, 10)          2 to the power 10 -> 1024.0
math.log(100, 10)        Log base 10 of 100 -> 2.0
math.pi                  Pi to 15 decimal places -> 3.141592653589793
```

Real use — ceil() for billing

Ola and Rapido charge per minute, rounding up — a 4.2-minute ride costs 5 minutes. math.ceil() is exactly the function behind that calculation. Any system that rounds up rather than to the nearest whole number — data storage, taxi fares, subscription billing — uses ceil.

```
duration_min = 4.2       Actual ride time
rate_per_min = 2.5       Rate in rupees per minute
```

```
billed = math.ceil(duration_min) * rate_per_min
```
ceil(4.2)=5, 5*2.5 = 12.5 rupees

TRY THIS LIVE

Use math.sqrt() to find the square root of 2, 3 and 5. Then use math.ceil() and math.floor() on each result. Notice how they differ from round() — they always go in one direction.

Function	What it returns	Real use
math.sqrt(x)	Square root of x	Distance, geometry, ML models
math.ceil(x)	Nearest integer >= x	Billing, storage allocation
math.floor(x)	Nearest integer <= x	Age from float, page counts
math.pow(x, y)	x to the power y (float)	Compound interest, exponents
math.log(x, base)	Logarithm of x in given base	Signal strength, data compression
math.pi	3.14159... constant	Circle area, angles
math.e	2.71828... constant	Exponential growth, finance

02 The random Module

The random module generates pseudorandom numbers — numbers that behave randomly but are produced by a mathematical formula. It is used everywhere: shuffling quiz questions, generating OTPs, picking a winner from a lucky draw, and setting starting conditions in simulations.

```
import random                          Load the module
random.randint(1, 100)                 Random integer between 1 and 100 inclusive
random.random()                        Random float between 0.0 and 1.0
random.uniform(10.0, 20.0)             Random float between 10.0 and 20.0
random.choice(['A', 'B', 'C', 'D'])    Pick one item at random
random.choices([1,2,3,4], k=3)         Pick 3 with replacement
random.sample(range(1,50), 6)          Pick 6 unique numbers (like a lottery)
```

Shuffling a quiz

```
questions = ['Q1', 'Q2', 'Q3', 'Q4', 'Q5']   Original question order
random.shuffle(questions)                     Shuffles the list IN PLACE — no return value
print(questions)                              Different order every run
```

03 Seeding — Making Random Reproducible

By default, random uses the system clock as its seed — so results differ every run. Setting a fixed seed with random.seed() forces the same sequence every time. This is essential in machine learning so experiments can be reproduced exactly.

```
random.seed(42)                       Fix the starting point
print(random.randint(1, 100))         Same number every run when seed is 42
random.seed(42)                       Reset the seed
print(random.randint(1, 100))         Exact same number again
```

PRO TIP — random is not secure enough for passwords

random is fine for games, simulations and quiz shuffles. For OTPs, tokens or anything security-related, use the secrets module instead — it uses a cryptographically strong source. Writing secrets.randbelow(1000000) gives a 6-digit OTP that is safe to use in production.

CASE STUDY — random_state=42 and reproducible ML

Context — Scikit-learn, the most widely used Python machine learning library, uses random numbers internally when splitting data, initialising models and sampling.

Challenge — Machine learning results can change every run if random operations produce different sequences. Researchers must be able to reproduce their exact results to verify and publish findings.

What they did — Almost every scikit-learn function that uses randomness accepts a random_state parameter — effectively a seed. Setting random_state=42 became a community convention for reproducible experiments across the world.

Result — The number 42 appears in millions of data science notebooks and tutorials globally — all because of one seed parameter that wraps Python's random module.

Takeaway — Understanding random.seed() is the same skill behind reproducible machine learning experiments.

Source: scikit-learn documentation (scikit-learn.org/stable/glossary.html#term-random_state)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms Using Python (Prof. Madhavan Mukund, CMI) — lectures covering standard library modules and built-in tools

Link — https://nptel.ac.in/courses/106106145

"Python's math module took decades of mathematical work and wrapped it in one import."

WHERE TO GO NEXT

You can now use Python's standard library for professional-grade maths and randomness.

Explore deeper — Browse the full math module at docs.python.org/3/library/math.html — many functions there will surprise you with their usefulness.

Try on your own — Build a mini OTP generator: use random.randint(100000, 999999) to generate a 6-digit code, print it, then try the same thing using secrets.randbelow(900000) + 100000.

Connect the dots — Next session we group modules into packages — learning how to organise a whole project into folders with __init__.py.

Going further — Explore the statistics module (mean, median, stdev) — it complements math for data analysis tasks without needing to install anything.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. In simple Hinglish, explain the difference between math.ceil(), math.floor() and round() using a rickshaw fare example — show when each one gives a different answer. Keep it under 150 words and include one code snippet.

SESSION
12

60 MIN · FUNCTIONS
Packages

Learning Objectives

- Explain the difference between a module and a package
- Create a package folder with a correct __init__.py file
- Import from a nested package using dot notation
- Use __init__.py to control what a package exposes
- Understand how third-party packages like NumPy and Django are structured
- Lay out a clean folder structure for a real multi-module project

01 Module vs Package

A module is one .py file. A package is a folder of modules — a way to group related files under one namespace. If TOPS has a training department with separate teams for IT, Design and Data, a package would be the department folder and each module would be one team's file.

CONCEPT — What makes a folder a package

A regular folder becomes a Python package the moment it contains a file called __init__.py. That file can be completely empty — its presence is the signal Python needs to treat the folder as importable. Without it, Python ignores the folder.

Module

A single .py file. import student loads student.py. Good for small, self-contained tools.

Package

A folder with __init__.py and one or more .py files inside. import tops.student loads the student module from inside the tops package.

02 Creating Your First Package

Build the folder, add __init__.py, then add your module files. The folder name becomes the package name — keep it lowercase, no spaces or hyphens.

Folder structure

```
project/          Your project root
+-- main.py       Entry point — runs the program
+-- tops/         The package folder
```

```
    +-- __init__.py     Makes 'tops' a package (can be empty)
    +-- student.py      Student-related functions
    +-- results.py      Results and grading functions
```

- **tops/student.py**

```
def greet(name):
    return f'Welcome to TOPS, {name}!'
def enroll(name, course):
    return f'{name} enrolled in {course}.'
```

- **main.py — importing from the package**

```
from tops import student                Import the student module from the tops package
print(student.greet('Ravi'))            Welcome to TOPS, Ravi!
from tops.student import enroll         Import one specific function
print(enroll('Priya', 'Django'))        Priya enrolled in Django.
```

> **TRY THIS LIVE**
>
> Create the tops/ package folder, add __init__.py (empty), add student.py with two functions. Then run main.py and import from the package. Confirm both import styles work.

03 Using __init__.py to Control Exports

__init__.py does not have to be empty. Whatever you import or define inside it becomes available directly on the package. This lets you create a clean public interface — callers import from tops, not from tops.student.

```
# tops/__init__.py                      Decide what tops exposes publicly
from .student import greet, enroll      The dot means 'from this package'
from .results import get_grade          Expose results function too
```

```
# main.py — now callers use just 'tops'
import tops                                 One import gets everything
print(tops.greet('Amit'))                   Works — exposed through __init__.py
```

04 Nested Packages (Sub-packages)

Packages can contain other packages. Every sub-folder with its own __init__.py becomes a sub-package. Django uses this deeply — django.contrib.auth is a sub-package inside django.contrib, which is itself a sub-package inside django.

```
tops/
+-- __init__.py
+-- training/              Sub-package
|   +-- __init__.py
|   +-- python_course.py
+-- placement/             Another sub-package
    +-- __init__.py
    +-- job_board.py
```

```
from tops.training import python_course          Access deeply nested module
from tops.placement.job_board import apply       Access a specific function
```

PRO TIP — Use relative imports inside a package

Inside a package, use relative imports (from .student import greet) rather than absolute ones (from tops.student import greet). Relative imports keep the package portable — if you rename the package folder, only the package name in main.py needs updating, not every internal import.

Import style	Example	When to use
Absolute	from tops.student import greet	In main.py or outside the package
Relative	from .student import greet	Inside the package itself (in __init__.py or sibling modules)
Package alias	import tops as t	When package name is long or conflicts

CASE STUDY — NumPy is a package you already use

Context — NumPy, the foundational Python library for numerical computing, is used by virtually every data science and ML project in the world.

Challenge — Provide fast array operations, linear algebra, Fourier transforms and random sampling — far too much for a single file.

What they did — NumPy is structured as a package with sub-packages: numpy.linalg (linear algebra),

numpy.fft (Fourier), numpy.random (random sampling), numpy.testing (test tools). Each sub-package has its own __init__.py that exposes a clean API.

Result — A developer writes import numpy as np and gets access to the entire ecosystem through dot notation — exactly the pattern you built today.

Takeaway — Every Python library you will ever pip install is a package built with the same __init__.py structure you just learned.

Source: NumPy source code and documentation (numpy.org/doc/stable)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms Using Python (Prof. Madhavan Mukund, CMI) — lectures covering modules, packages and project organisation

Link — https://nptel.ac.in/courses/106106145

"A package is just a folder with a purpose — and one small file that tells Python to trust it."

WHERE TO GO NEXT

You can now organise any Python project into reusable, importable packages.

Explore deeper — Read about Python's official packaging guide at packaging.python.org — it shows how your package would be prepared for pip install.

Try on your own — Extend the tops package: add a placement/ sub-package with a job_board.py that contains a function to list available jobs. Import it from main.py using both absolute and relative styles.

Connect the dots — Next session we write list comprehensions — a powerful one-line pattern that replaces many common for loops cleanly.

Going further — Read about pyproject.toml and how modern Python packaging works — the file that tells pip everything it needs to install your package.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. In simple Hinglish, explain the difference between a module and a package using a college institute as the analogy — the institute is the package, each department is a module, and the admin office (__init__.py) decides who can enter and what they can access. Keep it under 150 words.

SESSION
13

60 MIN · COLLECTIONS

List Comprehension & Nested Lists

Learning Objectives

- Write a basic list comprehension to replace a for loop that builds a list
- Add an if condition to filter items inside a comprehension
- Use if-else inside a comprehension to transform items conditionally
- Create and access a 2D nested list (a list of lists)
- Write a nested list comprehension to flatten or transform a 2D structure
- Decide when a comprehension is cleaner than a loop and when it is not

01 What is a List Comprehension?

A list comprehension is a compact way to build a new list from an existing one — all in a single line. Instead of writing a for loop, an if check and an append, you describe the new list directly. The result is the same; the code is shorter and often easier to read.

CONCEPT — Comprehension syntax

[expression for item in iterable if condition]. Read it left to right: 'give me expression, for each item in iterable, but only if condition is true.' The if part is optional — leave it out and every item is included.

for loop version

```
squared = []
for x in range(1, 6):
squared.append(x * x)
```

Four lines to build one list.

Comprehension version

```
squared = [x * x for x in range(1, 6)]
```

One line, same result: [1, 4, 9, 16, 25]

```
marks = [78, 32, 92, 45, 65, 28, 88]          Original list
doubled = [m * 2 for m in marks]              Transform every item
passed = [m for m in marks if m >= 35]        Filter: keep only passing marks
names = ['ravi', 'priya', 'amit']
proper = [n.capitalize() for n in names]      ['Ravi', 'Priya', 'Amit']
```

TRY THIS LIVE

Given marks = [78, 32, 92, 45, 65, 28, 88], write two comprehensions: one that adds a 5-mark bonus to every

score, and one that keeps only scores above 60. Print both results.

02 if-else Inside a Comprehension

You can also use if-else to transform every item — not filter them. The syntax flips: the if-else goes before the for, because it is part of the expression, not a filter. This is the most common point of confusion, so read the rule carefully.

```
marks = [78, 32, 92, 45, 65]
# if-else BEFORE 'for' -> transforms every item
result = ['pass' if m >= 35 else 'fail' for m in marks]
   ['pass','fail','pass','pass','pass']
# if AFTER 'for' -> filters items (no else allowed)
passed = [m for m in marks if m >= 35]          [78, 92, 45, 65]
```

PRO TIP — The position rule

If-else BEFORE for = transform every item (both branches must exist). If AFTER for = filter items (keep or skip, no else). Mix them up and Python raises a SyntaxError. When in doubt, read the comprehension left to right: output first, source second, filter last.

03 Nested Lists — Lists of Lists

A nested list is a list whose items are themselves lists — a 2D grid structure. Think of a classroom seating chart: rows of seats, each row being a list. Or a subject-wise marksheet: each student is a list of marks per subject.

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]    3×3 grid
matrix[0]                                      First row -> [1, 2, 3]
matrix[1][2]                                   Row 1, column 2 -> 6
for row in matrix:                             Loop over rows
    print(row)                                 Prints each row list
```

◆ Real example — batch marks

```
batch = [[78, 92, 65], [88, 74, 90], [55, 60, 72]]
   3 students, 3 subjects each
for i, student_marks in enumerate(batch, 1):     Loop with student number
```

```
avg = sum(student_marks) / len(student_marks)
    Average per student
      print(f'Student {i}: {avg:.1f}')              Student 1: 78.3 ...
```

04 Nested List Comprehensions

A nested comprehension has two for clauses — one for the outer list and one for the inner. The most common use is flattening: turning a 2D list into a plain 1D list. Read the for clauses left to right — outer loop first, inner loop second.

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [n for row in matrix for n in row]          Flatten: [1,2,3,4,5,6,7,8,9]
evens = [n for row in matrix for n in row if n % 2 == 0]
    Flatten and filter: [2,4,6,8]
doubled = [[x * 2 for x in row] for row in matrix]
    Transform each cell: [[2,4,6],[8,10,12],...]
```

WARNING — Comprehension readability has a limit

A nested comprehension with a condition is often the most complex thing you should put in one line. Three or more for clauses, or complex expressions, make the code harder to understand than a plain loop. Python's own style guide (PEP 8) says: if a comprehension does not fit comfortably on one line, use a loop.

Pattern	Syntax	Use case
Basic	[expr for x in lst]	Transform every item
Filter	[expr for x in lst if cond]	Keep matching items only
Transform all	[a if cond else b for x in lst]	Label/convert every item
Flatten 2D	[x for row in grid for x in row]	Turn list of lists into one list
Transform 2D	[[f(x) for x in row] for row in grid]	Apply operation to every cell

CASE STUDY — Comprehensions were born from mathematics

Context — List comprehensions were added to Python 2.0 in 2000, inspired directly by Haskell's list comprehensions and mathematical set-builder notation.

Challenge — Python needed a clean way to express 'give me all x squared where x is in this set' — the kind of statement mathematicians write naturally but programmers had to express with verbose loops.

What they did — Guido van Rossum accepted PEP 202, which introduced the [expr for x in iterable if cond] syntax — matching how mathematicians write { x^2 | x in S, x > 0 } almost symbol for symbol.

Result — List comprehensions became one of Python's most loved features. Google's Python Style Guide explicitly approves them for simple cases, and they appear in virtually every serious Python codebase.

Takeaway — When you write a comprehension, you are using a feature borrowed directly from mathematics — making your code both shorter and more expressive.

Source: PEP 202 (peps.python.org/pep-0202) and Google Python Style Guide (google.github.io/styleguide/pyguide.html)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms Using Python (Prof. Madhavan Mukund, CMI) — lectures covering list comprehensions and nested data structures

Link — https://nptel.ac.in/courses/106106145

"A comprehension reads like a sentence: give me this, for each of those, if it passes this test."

WHERE TO GO NEXT

You can now replace most list-building loops with a single readable line.

Explore deeper — Read about dictionary and set comprehensions — the same syntax works for {k: v for ...} and {x for ...} too.

Try on your own — Given a 3×3 matrix of exam marks, use a nested comprehension to add a 5-mark bonus to every cell, then flatten the result into one list and find the overall average.

Connect the dots — Our final session builds on this directly — using loops and comprehension-style thinking to construct nested dictionaries dynamically.

Going further — Look into generator expressions — they use the same syntax as comprehensions but use () instead of [] and process items lazily, saving memory for huge datasets.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. In simple Hinglish, explain the difference between [x for x in lst if cond] and [a if cond else b for x in lst] using a cricket match scorecard as the example — one filters players who scored above 50, the other labels every player as 'hero' or 'duck'. Keep it under 150 words and show both lines of code.

SESSION
14

60 MIN · COLLECTIONS
Dynamic Nested Dictionaries

Learning Objectives

- Build a nested dictionary dynamically from a flat list of records using a loop
- Use setdefault() to safely initialise inner dictionaries without overwriting
- Use collections.defaultdict to remove the initialisation step entirely
- Group records by a key to create a lookup-style nested structure
- Access, update and iterate over a dynamically built nested dictionary
- Recognise this pattern as the foundation of grouping in data pipelines

01 The Problem — Data Arrives Flat, We Need It Grouped

Real data rarely arrives pre-grouped. A student export from a TOPS ERP gives you a flat list of rows — each row has a city, a course, a name and a score. To answer questions like 'who are all the Ahmedabad students?' or 'what is the average score per course?', you need to restructure that flat data into a nested dictionary dynamically, record by record.

> **CONCEPT — Dynamic nested dictionary**
>
> A dynamic nested dict is built in a loop, one record at a time. The outer key is usually a grouping field (city, course, category). The inner dictionary holds the details for each group. You do not know all the keys in advance — they are created as data arrives.

02 Method 1 — Manual if-check

The safest starting point: before adding to an inner dictionary, check whether the outer key already exists. If not, create it. If yes, just add to it. This works but involves extra lines on every insertion.

```
records = [                                    Flat list of student records
    ('Ravi',   'Python', 'Ahmedabad', 88),
    ('Priya',  'Django', 'Surat',     92),
    ('Amit',   'Python', 'Ahmedabad', 65),
    ('Pooja',  'Django', 'Ahmedabad', 79),
]
by_city = {}                                   The nested dict we will build
for name, course, city, score in records:      Unpack each record
    if city not in by_city:                    Create the inner dict if this city is new
        by_city[city] = {}
```

```
by_city[city][name] = {'course': course, 'score': score}
```
Add the student

```
print(by_city['Ahmedabad'])                    {'Ravi': {...}, 'Amit': {...}, 'Pooja': {...}}
print(by_city['Ahmedabad']['Ravi']['score'])   88
```

TRY THIS LIVE

Run the code above then add three more records with a new city. Print the complete by_city dictionary and count how many students are in each city by looping over by_city.items().

03 Method 2 — setdefault()

setdefault(key, default) does the if-check and creation in one line: if the key exists it returns the existing value; if not it inserts the default and returns it. This collapses the two-line if block into a single line without changing behaviour.

```
by_city = {}
for name, course, city, score in records:
by_city.setdefault(city, {})[name] = {'course': course, 'score': score}
```
One line does the check and the insert

CONCEPT — setdefault() return value

by_city.setdefault(city, {}) always returns the inner dict — whether it just created it or found an existing one. Chaining [name] = ... on that return value is what makes the one-liner work. It is not magic — just two operations chained.

04 Method 3 — defaultdict (the cleanest way)

collections.defaultdict takes a factory function. Whenever you access a key that does not exist, it calls that factory to create the default value automatically — no if-check, no setdefault needed. defaultdict(dict) is the standard tool for building nested dictionaries dynamically.

```
from collections import defaultdict          Import from the standard library
by_city = defaultdict(dict)                  Any missing key auto-creates an empty dict
for name, course, city, score in records:
```

```
by_city[city][name] = {'course': course, 'score': score}
    No check needed — defaultdict handles it
```

setdefault()

Works on a plain dict. No import needed. Slightly more explicit — good when you want the logic visible to a reader unfamiliar with defaultdict.

defaultdict(dict)

Cleanest syntax for dynamic nesting. Requires one import. Preferred in production code and data pipelines — the intent is clear from the type declaration alone.

05 Grouping by Multiple Keys

The same pattern scales to three or more levels of nesting — group by city, then by course within each city. Each level just needs its own setdefault or defaultdict layer.

```
from collections import defaultdict
by_city_course = defaultdict(lambda: defaultdict(list))
    3-level: city -> course -> list of students
for name, course, city, score in records:
by_city_course[city][course].append({'name': name, 'score': score})
    Add to the correct bucket
print(by_city_course['Ahmedabad']['Python'])     [{'name':'Ravi','score':88},{'name':'Amit','score':65}]
```

PRO TIP — Convert defaultdict before sharing

defaultdict prints with its type name and can confuse readers unfamiliar with it. Before logging, returning from a function or storing to JSON, convert it: dict(by_city). For deeply nested structures use a recursive conversion or json.dumps() directly.

06 Computing Aggregates from the Nested Structure

Once the data is grouped, computing summaries — average score per city, count per course — is a natural loop over the nested structure using .items().

```
for city, students in by_city.items():          Outer loop over cities
scores = [s['score'] for s in students.values()]
    Comprehension: collect all scores
    avg = sum(scores) / len(scores)             Average for this city
```

```
print(f'{city}: {len(scores)} students, avg {avg:.1f}')
```

Ahmedabad: 3 students, avg 77.3

CASE STUDY — Every JSON API response is a dynamic nested dict

Context — When apps like Zomato, Swiggy or BookMyShow fetch data from a server, the server replies in JSON — and Python's json module turns that directly into a nested dictionary.

Challenge — A restaurant listing must carry nested data: restaurant details, a list of menu sections, and items within each section — all in one structured response.

What they did — The server builds this nested structure dynamically while querying the database — grouping items by category, attaching them to their restaurant — exactly the loop-and-nest pattern you built today. Python's json.dumps() then serialises it for the network.

Result — Every API call your future Django app makes or receives will be a nested dictionary in disguise. Reading and writing them is the same skill you practised today.

Takeaway — Dynamic nested dictionaries are not an advanced topic — they are the shape of the real world's data.

Source: Python docs — json module (docs.python.org/3/library/json.html)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms Using Python (Prof. Madhavan Mukund, CMI) — lectures covering dictionaries and data grouping patterns

Link — https://nptel.ac.in/courses/106106145

“Flat data is easy to collect. Nested data is easy to query. The loop in between is your job.”

WHERE TO GO NEXT

You can now take any flat list of records and reorganise it into a structured, queryable nested dictionary.

Explore deeper — Read about itertools.groupby — a standard library tool that groups a sorted iterable the same way, without building the dict manually.

Try on your own — Take the student records from this session and group them by course instead of city. Then compute the average score per course and find the highest-scoring course.

Connect the dots — This is the final session of Module 2. You are now ready for Module 3 — Advanced Python, starting with file handling, where structured dictionaries go in and out of real files.

Going further — Explore pandas GroupBy — the professional tool that performs this same group-and-aggregate pattern on millions of rows in milliseconds, built on the same dictionary logic you mastered today.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. In simple Hinglish, explain how to group a flat list of student records by city using a defaultdict, using a school assembly where the teacher groups students city-wise as the analogy. Show the key three lines of code and explain what each does. Keep it under 150 words.

REVIEW
?

INTERVIEW PREP · MODULE 2

Interview Questions

◆ Collections, Functions and Modules in Python

Rehearse each answer out loud and aim to deliver it within 60 seconds. The hint is only a nudge — attempt the answer first, then compare with the model answer below.

1 **What is a Python list, and how can it store values of different data types?**

Think about how lists can hold integers, strings, floats, etc. together.

MODEL ANSWER
A list is an ordered, mutable collection defined with square brackets. Unlike arrays in some languages, Python lists can hold elements of different data types in the same collection — for example, mixed = [1, 'hello', 3.14, True]. Each element is stored at a numbered index starting from 0, and the list grows or shrinks dynamically.

2 **How would you update the value at a specific index in a list?**

MODEL ANSWER
You assign a new value directly to the index: nums = [10, 20, 30]; nums[1] = 99 changes the second element to 99, giving [10, 99, 30]. Because lists are mutable, index assignment modifies the list in place without creating a new one.

3 **Describe two ways to remove an element from a list in Python.**

Consider both value-based and index-based removal.

MODEL ANSWER
remove(value) searches for the first occurrence of a given value and deletes it — fruits.remove('banana') removes 'banana' by value. pop(index) removes the element at a specific index and returns it — fruits.pop(0) removes and returns the first item. Use remove() when you know the value and pop() when you know the position.

4 **How does a 'for' loop help you iterate through all elements in a list?**

MODEL ANSWER
A for loop automatically picks each element from the list one at a time and assigns it to a loop variable. For example, for item in my_list: print(item) prints every element without needing to manage an index counter. This is clean and readable and is the idiomatic Python way to process every item in a collection.

5 **What is the purpose of using 'range' with lists in a for loop?**

Think about what 'range' provides compared to directly looping over the list.

MODEL ANSWER

range() generates a sequence of numbers, usually used as indices. for i in range(len(my_list)) gives you the index at each step, so you can access both the index and the value — my_list[i]. This is useful when you need the position, want to modify elements in place, or need to loop with a step. For read-only iteration, looping directly over the list is preferred.

6 **Write a Python loop to print each element in the list ['a', 3, 5.5, 'hello'] along with its index.**

MODEL ANSWER

```
items = ['a', 3, 5.5, 'hello']
for i, val in enumerate(items):
print(i, val)
```

enumerate() is the cleanest way to get both index and value together. It returns pairs like (0, 'a'), (1, 3), etc., avoiding the need for a separate counter variable.

7 **What does the built-in function 'round()' do when applied to a list of float numbers?**

Consider if 'round' works directly on lists or if you need to use it differently.

MODEL ANSWER

round() works on a single number, not on an entire list — calling round(my_list) raises a TypeError. To round every float in a list you apply it element by element, typically with a list comprehension: rounded = [round(x, 2) for x in floats]. This gives you a new list with each value rounded to the specified decimal places.

8 **Explain the difference between 'sort()' and 'sorted()' when working with lists in Python.**

One modifies the list in place, the other returns a new list.

MODEL ANSWER

sort() is a list method that sorts the list in place and returns None — it changes the original list directly. sorted() is a built-in function that returns a new sorted list and leaves the original unchanged. Use sort() when you do not need the original order, and sorted() when you want to keep the original list intact or sort any other iterable, not just a list.

9 **How would you use the 'zip()' function to combine two lists into a list of tuples?**

MODEL ANSWER

```
names = ['Ravi', 'Sita']
marks = [85, 92]
paired = list(zip(names, marks))
print(paired)  # [('Ravi', 85), ('Sita', 92)]
```

zip() pairs elements at matching indices from both lists into tuples. Wrapping it in list() converts the zip object into a regular list. If the lists have unequal lengths, zip() stops at the shorter one.

10 **What is a tuple, and how is it different from a list?**

Focus on mutability.

MODEL ANSWER
A tuple is an ordered, immutable collection defined with parentheses: coords = (10, 20). The key difference from a list is that you cannot change, add, or remove elements after creation. Tuples are faster and use less memory than lists, making them ideal for fixed data such as coordinates, database records, or function return values where the content should not change.

11 **How do you create a tuple that contains both strings and numbers?**

MODEL ANSWER
You define it with parentheses, mixing types freely: profile = ('Ravi', 21, 8.5) stores a string, integer, and float in a single tuple. Python tuples can hold any mix of data types just like lists. Access elements by index — profile[0] gives 'Ravi'.

12 **Explain how slicing works for tuples. Give an example.**

MODEL ANSWER
Tuple slicing uses the same syntax as list slicing: t[start:end:step], where start is inclusive and end is exclusive. For example, t = (10, 20, 30, 40, 50); t[1:4] returns (20, 30, 40). Slicing always produces a new tuple and does not modify the original. Negative indices and step values work the same as with strings and lists.

13 **How can you convert a tuple to a list and vice versa in Python?**

There are built-in functions for both conversions.

MODEL ANSWER
Use list() to convert a tuple to a list: my_list = list(my_tuple). Use tuple() to convert a list to a tuple: my_tuple = tuple(my_list). A common pattern is to convert a tuple to a list when you need to modify it, make your changes, then convert it back to a tuple to restore immutability.

14 **How do you update the value for a specific key in a Python dictionary?**

MODEL ANSWER
Assign a new value directly to the key: student = {'name': 'Ravi', 'age': 20}; student['age'] = 21 updates age to 21. If the key exists it is overwritten; if it does not exist a new key-value pair is added. Alternatively, student.update({'age': 21}) does the same thing and also supports updating multiple keys at once.

15 **What is a nested dictionary? Give a simple example.**

MODEL ANSWER
A nested dictionary is a dictionary where one or more values are themselves dictionaries. For example: students = {'Ravi': {'age': 20, 'marks': 85}, 'Sita': {'age': 21, 'marks': 92}}. You access inner values by chaining keys: students['Ravi']['marks'] returns 85. Nested dictionaries are useful for representing structured records like student profiles or JSON data.

16 **Given two lists: keys = ['name', 'age'], values = ['Ravi', 21], write code to create a dictionary mapping keys to values using 'zip()'.**

MODEL ANSWER

```
keys = ['name', 'age']
values = ['Ravi', 21]
result = dict(zip(keys, values))
print(result)  # {'name': 'Ravi', 'age': 21}
```

zip() pairs each key with its corresponding value, and dict() converts those pairs into a dictionary. This is a clean, Pythonic way to build a dictionary from two parallel lists.

17 **Show how to create a dictionary from two lists using a loop instead of 'zip()'.**

MODEL ANSWER

```
keys = ['name', 'age']
values = ['Ravi', 21]
result = {}
for i in range(len(keys)):
result[keys[i]] = values[i]
print(result)  # {'name': 'Ravi', 'age': 21}
```

The loop uses index i to access matching elements from both lists and assigns them as key-value pairs. This is more verbose than zip() but makes the logic explicit and is easy to follow.

18 **How would you count the frequency of each character in a string using a dictionary?**

Consider iterating over each character and updating a count.

MODEL ANSWER

```
text = 'hello'
freq = {}
for ch in text:
freq[ch] = freq.get(ch, 0) + 1
print(freq)  # {'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

get(ch, 0) returns the current count for that character, or 0 if it has not been seen before. Adding 1 each time a character appears builds up an accurate frequency map.

19 **Write a Python program that counts the number of words in a given sentence.**

MODEL ANSWER

```
sentence = input('Enter a sentence: ')
words = sentence.split()
print('Word count:', len(words))
```

split() breaks the string at whitespace and returns a list of words. len() then counts the elements. This handles multiple consecutive spaces correctly because split() without arguments treats any amount of whitespace as a single delimiter.

20 **What is the difference between positional and keyword arguments in Python functions?**

Think about how arguments are matched to parameters.

MODEL ANSWER

Positional arguments are matched to parameters by their order in the function call — the first value goes to the first parameter. Keyword arguments are matched by name, so you explicitly write parameter_name=value, and order does not matter. Keyword arguments make calls more readable and allow you to skip optional parameters with defaults.

21 **How can you set a default value for a function argument in Python?**

MODEL ANSWER

Assign a default value to the parameter in the function definition: def greet(name='Guest'): print('Hello', name). If the caller does not pass that argument, the default is used. Parameters with defaults must come after parameters without defaults in the function signature.

22 **Write a function that takes two numbers and returns their sum, but if only one number is given, the second should default to 10.**

MODEL ANSWER

```
def add(a, b=10):
return a + b

print(add(5))     # 15
print(add(5, 3))  # 8
```

Setting b=10 as the default means the function works with one or two arguments. When called with one argument, b defaults to 10; when called with two, the passed value overrides the default.

23 **What is a lambda function in Python, and when would you use one?**

Consider situations where you need a quick, simple function.

MODEL ANSWER

A lambda is a small, anonymous function written in a single line: lambda arguments: expression. It is best used for short throwaway logic passed directly into another function, such as a sort key or a filter condition. For anything requiring multiple lines or reuse, a regular def function is clearer and more maintainable.

24 **Write a lambda function that takes two arguments and returns their product.**

MODEL ANSWER

```
multiply = lambda a, b: a * b
print(multiply(4, 5))  # 20
```

The lambda takes two parameters a and b, and the expression a * b is evaluated and returned automatically. This is equivalent to writing def multiply(a, b): return a * b, but in one line.

25 **How can you use a lambda function to sort a list of tuples by the second element in each tuple?**

MODEL ANSWER

```
data = [('Ravi', 85), ('Sita', 92), ('Arjun', 78)]
data.sort(key=lambda x: x[1])
print(data)  # sorted by marks ascending
```

The key argument tells sort() which value to compare. The lambda receives each tuple x and returns its second element x[1], so the list is sorted by marks rather than by name.

26 **How do you create a custom Python module (e.g., student.py) and use it in another file?**

Think about file creation and the import statement.

MODEL ANSWER

Create a file called student.py with functions or variables defined in it, for example def get_name(): return 'Ravi'. In another file in the same directory, write import student and call student.get_name(). Alternatively, use from student import get_name to import just that function directly. Python treats any .py file as a module.

27 **If you have a function defined in student.py, how do you call it from your main Python file?**

MODEL ANSWER

After importing the module with import student, you call the function as student.function_name(). If you used from student import function_name, you call it directly as function_name(). The first approach is safer for avoiding name conflicts; the second is more convenient when you only need one or two specific functions.

28 **Name two functions from the 'math' module and describe what they do.**

MODEL ANSWER

math.sqrt(x) returns the square root of x — for example, math.sqrt(16) returns 4.0. math.ceil(x) rounds a number up to the nearest integer — math.ceil(4.2) returns 5. The math module provides many mathematical functions and constants (like math.pi) ready to use without writing the logic yourself.

29 **How would you generate a random integer between 1 and 100 in Python?**

Look into the 'random' module.

MODEL ANSWER

```
import random
num = random.randint(1, 100)
print(num)
```

randint(a, b) returns a random integer N such that a <= N <= b — both endpoints are inclusive. The random module also has randrange() (exclusive upper bound) and random() (float between 0 and 1) for other use cases.

30 **What is the purpose of the '__init__.py' file when creating a Python package?**

Think about what makes a folder a package.

MODEL ANSWER

An __init__.py file signals to Python that a folder should be treated as a package rather than a plain directory. It can be empty or contain initialisation code that runs when the package is imported. Without it, Python 2 would not recognise the folder as importable; in Python 3 it is technically optional for namespace packages but still considered best practice.

31 **Explain how to structure a basic Python package with multiple modules.**

MODEL ANSWER

Create a folder (e.g., mypackage/) with an __init__.py file inside it. Place your module files alongside it — for example, student.py and teacher.py. In other scripts you can then import them as from mypackage import student or from mypackage.student import get_name. This structure keeps related code organised and makes it easy to reuse across projects.

32 **Write a list comprehension to create a list of squares of numbers from 1 to 5.**

MODEL ANSWER

```
squares = [x ** 2 for x in range(1, 6)]
print(squares)  # [1, 4, 9, 16, 25]
```

A list comprehension is a concise one-liner that replaces a for loop and append pattern. range(1, 6) generates 1 through 5 (6 is excluded), and x ** 2 computes the square of each number.

33 **How would you use a nested list comprehension to flatten a 2D list into a 1D list?**

Think about looping over sublists and their elements.

MODEL ANSWER

```
matrix = [[1, 2], [3, 4], [5, 6]]
flat = [item for row in matrix for item in row]
print(flat)  # [1, 2, 3, 4, 5, 6]
```

The outer loop iterates over each row, and the inner loop iterates over each item in that row. Reading left to right mirrors how you would write the equivalent nested for loops.

34 **Describe how to create a dynamic nested dictionary in Python, such as for storing student marks in multiple subjects.**

Consider using dictionaries inside dictionaries.

MODEL ANSWER

Start with an outer dictionary keyed by student name, and set the value for each student to another dictionary of subject-mark pairs: records = {'Ravi': {'Maths': 88, 'Science': 75}}. To add a new student dynamically, assign a new key: records['Sita'] = {}. Then add subjects one at a time: records['Sita']['English'] = 92. This structure mirrors a real-world table with rows and columns.

35 **Write code to add a new subject and mark for a student in an existing nested dictionary.**

MODEL ANSWER

```
records = {'Ravi': {'Maths': 88, 'Science': 75}}
records['Ravi']['English'] = 90
print(records)
# {'Ravi': {'Maths': 88, 'Science': 75, 'English': 90}}
```

Chaining two key lookups — first the student name, then the subject — navigates into the inner dictionary and adds or updates the entry directly.

36 **How can you use AI tools like ChatGPT or GitHub Copilot to help you write or debug Python code, and what should you always check before using their suggestions?**

Think about code generation and the importance of reviewing output for correctness.

MODEL ANSWER

You can describe a function requirement to ChatGPT or let Copilot autocomplete code as you type. Before using the output you should read through it to understand what it does, test it with your own data including edge cases, and check that variable names and logic match your project. AI tools can produce plausible-looking but incorrect code, so treat every suggestion as a draft that needs your review.

37 **Imagine you need to quickly generate a function to count word frequencies in a text. How could you use an AI assistant to speed up your workflow, and how would you verify the code works as expected?**

MODEL ANSWER

Prompt the AI with a clear requirement: 'Write a Python function that takes a string and returns a dictionary of word frequencies.' Review the generated code to check the logic, then run it with a simple test like count_words('hello world hello') and verify the output is {'hello': 2, 'world': 1}. Also test with empty strings and punctuation to catch edge cases the AI may have missed.

38 **Integrative: How would you write a function that takes a list of sentences, counts the number of words in each sentence, and returns a dictionary mapping each sentence to its word count?**

Combine knowledge of functions, lists, and dictionaries.

MODEL ANSWER

```
def word_counts(sentences):
return {s: len(s.split()) for s in sentences}

result = word_counts(['Hello world', 'Python is great'])
# {'Hello world': 2, 'Python is great': 3}
```

The dictionary comprehension iterates over the sentences list, uses split() to tokenise each sentence, and len() to count words. Each sentence becomes a key and its word count becomes the value.

39 **Integrative: Given a list of student names and a list of their marks, write code to create a dictionary mapping each student to their mark, and then use a function to find the student with the highest mark.**

You'll need to use zip(), dictionaries, and functions together.

MODEL ANSWER

```
names = ['Ravi', 'Sita', 'Arjun']
marks = [85, 92, 78]
records = dict(zip(names, marks))

def top_student(d):
return max(d, key=d.get)

print(top_student(records))  # Sita
```

zip() pairs names with marks, dict() builds the mapping, and max() with key=d.get compares by mark values and returns the name of the highest scorer.

40 **Integrative: How can you use a lambda function along with the 'sorted()' function to sort a list of dictionaries representing students by their 'age' key?**

MODEL ANSWER

```
students = [{'name': 'Ravi', 'age': 22}, {'name': 'Sita', 'age': 20}, {'name': 'Arjun', 'age': 21}]
sorted_students = sorted(students, key=lambda s: s['age'])
print(sorted_students)
```

The lambda extracts the 'age' value from each dictionary, and sorted() uses those values to determine order. Adding reverse=True would sort in descending age order.